

COSC 2230 Augustana Univ Fall 26

Immutability

- When is a data type called "immutable"?

A data type is **immutable** if its objects cannot change value once created. "hello" is a value of an immutable type (String); an int[] array is a value of a mutable type (mutated via array index).

The one question immutability answers: *can another reference change what I am holding?*

- Let's look at three cases.
 - **Immutable (String)** - the callee cannot touch the caller's value:

```
public class ImmutableString {
    static void shout(String s) { s = s.toUpperCase(); } // new object, not a
    public static void main(String[] args) {
        String greeting = "hello";
        shout(greeting);
        System.out.println(greeting); // hello
    }
}
```

Java passes arguments by value, including references: s is a fresh local variable that gets a copy of the reference in greeting. s = s.toUpperCase() rebinds that local copy to point at a different object. greeting back in main is untouched - it still points where it always did. To "shout", point greeting at the new object:

```
greeting =
```

- **Mutable (array)** - change through one reference, see it through the other:

```
import java.util.Arrays; // Not sure students have to add this?
public class MutableArray {
    static void zero(int[] a) { a[0] = 0; }
    public static void main(String[] args) {
        int[] data = {1, 2, 3}; // create object on the heap and stores reference
        zero(data); // copies value of reference into parameter a (same heap object)
        System.out.println(Arrays.toString(data)); // [0, 2, 3]
    }
}
```

The array container is mutable here, not the `int` elements. You're overwriting slot 0. `zero()` is a setter reaching into your data: that's what you need a "defensive copy" for - copy the array on the way in so no outside reference is aliased to your internal state.

- **Primitive** - nothing to alias; the callee gets a copy of the value:

```
public class Primitive
{
    static void bump(int x) { x++; }

    public static void main(String[] args)
    {
        int i = 0;
        bump(i);
        System.out.println(i); // 0
    }
}
```

In `main`, `i` holds the `int 0`. `bump(i)` copies that value into the parameter `x`, an independent local variable, not a link back to `i`. `x++` bumps that copy to 1, but the copy is discarded when `bump` returns, and `void` means nothing is handed back. So `i`, never reassigned and never connected to `x`, still prints 0. With a primitive **there is no shared heap object to mutate** and no reference to **alias**, so a method can affect the caller's variable only through a returned value the caller assigns.

To return 1, you'd have to return the new value and have `main` assign it back:

```
static int bump(int x) { return x + 1; }

public static void main(String[] args)
{
    int i = 0;
    i = bump(i);           // reassign with the returned value
    System.out.println(i); // 1
}
```

- Summary:

Type	Example	Can a callee/alias change what you hold?
String, primitives, wrappers	"hello", int	no
arrays, ArrayList, most objects	int[], StringBuilder	yes - shared reference

- **Wrapper (Integer)** - autoboxing converts `int <-> Integer`, and the wrapper is immutable, so it is as safe as String:

```
public class Wrapper
{
    static void bump(Integer n) { n = n + 1; } // unbox, add, re-box: new object
    public static void main(String[] args)
    {
        Integer count = 5;           // autoboxing: int 5 -> Integer
        bump(count);
        System.out.println(count); // 5 - unchanged
        int prim = count;           // unboxing: Integer -> int
        System.out.println(prim); // 5
    }
}
```

```
}  
}
```

- Python has the same split: immutable `int` `float` `str` `tuple` `bytes`, mutable `list` `dict` `set`. With `b = a`; `b.append(4)` the change is visible through `a` too; `a = (1, 2, 3)` (a tuple) cannot be mutated.
- Why is this important for algorithms?
 1. Sorting/Searching in-place algorithms only make sense because arrays are mutable.
 2. Stacks: `Stack<Integer>` autoboxing only works because `Integer` is immutable.
 3. Symbol tables: hash-based and comparison-based symbol table keys need to be immutable (or at least not mutated while in the table).

The final modifier, aliasing bugs, and defensive copy

- To enforce immutability in a data type, you can label it as `final`. A `final` variable is assigned a value only once, either in an inline initialization statement, or in the constructor. Any other code that could modify the value leads to a compile-time error.
- The `final` modifier guarantees immutability only for primitive type instance variables but not for reference types: if an instance variable of a reference type has the `final` modifier, its value (the object reference) will never change - it will always refer to the same object. However the value of the object itself can change.
- If your `final` instance variable is an array, you cannot change the array's length or type, but you can change the values of the array elements. This leads to **aliasing bugs**.
- Example: in the `Vector` class below, `final` only freezes the reference - after the constructor, `cartesian` can never execute again, but if the constructor stores the caller's array by reference instead of copying it, the object and the caller are both aliased to the same array on the heap, and external mutation leaks through to the supposedly immutable type.

```
public class VectorBug  
{  
    private final double[] cartesian;  
    public VectorBug(double[] data) { cartesian = data; } // bug 1: stores  
    // reference,  
    // not a copy  
    public double component(int i) { return cartesian[i]; }  
    public double[] cartesian() { return cartesian; } // bug 2: leaks  
    // internal  
    // reference  
  
    public static void main(String[] args)  
    {  
        double[] coords = {4.0, 3.0};  
        VectorBug v = new VectorBug(coords);  
        System.out.println(v.component(0));  
        // caller still holds the same array object  
        coords[0] = 999.0;  
        // bug 1: 999.0 - v changed with no mutator ever called  
        System.out.println(v.component(0));  
        // bug 2: mutates v's internal state with no setter  
        v.cartesian()[0] = -1.0;  
        System.out.println(v.component(0));  
    }  
}
```

- In the code example, the object `v` has no setters, `cartesian` is `final`, and yet its state changed: two references, one array (bug 1). Mutating through either is indistinguishable to Java. The same hole exists on the way out, if an accessor hands back the live array (bug 2).
- The fix: **defensive copy** at both boundaries - copy on the way in (constructor), and on the way out (any accessor that returns the array) using `data.clone()` (or `Arrays.copyOf()`).

```
public class Vector
{
    private final double[] cartesian;
    public Vector(double[] data)
    {
        cartesian = data.clone(); // alternatively:
        // Arrays.copyOf(data, data.length)
    }
    public double component(int i) { return cartesian[i]; }
    public double[] cartesian()
    {
        return cartesian.clone();
    }

    public static void main(String[] args)
    {
        double[] coords = {3.0, 4.0};
        Vector v = new Vector(coords);
        System.out.println(v.component(0));
        // caller still holds the same array object
        coords[0] = 999.0;
        // bug 1 fixed: No access
        System.out.println(v.component(0));
        // bug 2 fixed: No access
        v.cartesian()[0] = -1.0;
        System.out.println(v.component(0));
    }
}
```

- Note that `cartesian` is a private array that only `Vector` itself ever writes to: no outside reference holds a live reference to it in either direction, so `final` actually delivers immutability.
- In the book's `Vector` class, the constructor copies the vector element-wise (rather than with a library call), and the accessor is left out altogether.
- See also: difference between **shallow** and **deep copy** (same thing here, for copy of primitive double content types).
- The textbook code differs in two ways:
 1. the defensive copy is hand-rolled, spelled out loop by loop, rather than using a library function,
 2. The API only exposes `cartesian(int i)`, and never the live `coords` array itself so there is no "bug 2" to defend against.
- This code

```
public class Vector
{
    private final double[] cartesian;
    public Vector(double[] data)
    {
        // defensive copy on the way in (construction)
        cartesian = new double[data.length];
        for (int i = 0; i < data.length; i++)
            cartesian[i] = data[i];
    }
}
```

```

public double component(int i) {return cartesian[i];}
public double[] cartesian()
{    // defensive copy on the way out (access)
double[] copy = new double[cartesian.length];
for (int i = 0; i < cartesian.length; i++)
    copy[i] = cartesian[i];
return copy;
}

public static void main(String[] args)
{
double[] coords = {4.0,3.0};
Vector v = new Vector(coords);
System.out.println(v.component(0));
// caller still holds the same array object
coords[0] = 999.0;
// bug 1 fixed: no access to `cartesian`
System.out.println(v.component(0));
// bug 2 fixed: no direct access
v.cartesian()[0] = -1.0;
System.out.println(v.component(0));
}
}

```

Author: Marcus Birkenkrahe

Created: 2026-09-09 Wed 20:49